

Programmation Concurrente: Thread (Suite)

CPE Lyon

2026

Consignes générales:

- Vos programmes doivent être compilés avec les options: -Wall -Wextra -g
- Vos programmes ne doivent pas avoir de warnings
- Testez plusieurs fois de suite vos programmes et vérifiez que les résultats obtenus sont ceux attendus.

Exercice 1: Gestionnaire de Billes

On souhaite réaliser l'exemple suivant:

- **N** threads (par ex. $N = 4$) ont besoin chacun d'un nombre **k** d'une ressource (par ex. des Billes) pour avancer leur travail
- Cette ressource existe en un **nombre limité** : on ne peut satisfaire la demande de tout le monde en même temps. Par exemple, la demande des N ($N=4$) est de (4, 3, 5, 2) billes et on ne dispose que de **nb_max_billes** = 9 billes
- Chaque thread répète la séquence (par ex. m fois) : "demander k ressources, utiliser ressources, rendre k ressources"
- Le "main" crée les N threads. Il crée également un thread contrôleur qui vérifie en permanence si le nombre de Billes disponible est dans l'intervalle $[0..nb_max_billes]$
- Pour chaque thread T_i , l'accès à la ressource se fait par une fonction "demander(k)" qui doit bloquer le demandeur tant que le nombre de billes disponible est inférieur à k
- T_i , rend les k billes acquises après son travail et recommence sa séquence

Algorithme

Main:

```
Créer Contrôleur
Lancer Contrôleur
Créer N threads travailleurs
Attendre la fin des N threads
Terminer le Contrôleur
```

Travailleur:

```
itérer m fois
    Demander(k_billes)
    simuler un travail avec une attente aléatoire (sleep)
    Rendre(k_billes)
```

Demander:

```
Tant que nbr_billes_disponible < k_billes:
    se bloquer sur un semaphore d'attente
nbr_billes_disponible = nbr_billes_disponible - k_billes
```

Rendre:

```
nbr_billes_disponible = nbr_billes_disponible - k_billes  
réveiller ceux qui sont en attente (cf Demander)
```

Contrôleur:

```
Iterer toujours  
    Verifier que  $0 \leq \text{nbr\_billes\_disponible} \leq \text{max\_billes}$   
    delai(1 sec)
```

Remarques

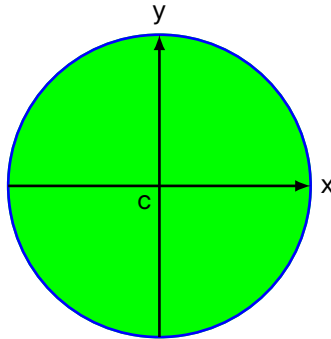
nbr_billes_disponible est une ressource partagée il faudra donc évidemment protéger les accès en lecture et en écriture.

Quand les threads en attente sont réveillés, ça ne veut pas dire que le nombre de billes dont ils ont besoin sont disponibles mais simplement qu'un thread a posé des billes et qu'il est possible qu'ils en aient suffisamment pour travailler. Si ce n'est pas le cas, ils vont se remettre en attente.

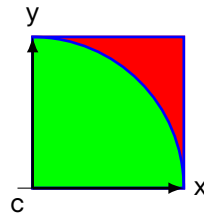
L'usage du semaphore pour l'attente est là pour garantir une attente passive. Il ne sert à rien de regarder le nombre de billes disponible tant qu'un thread n'en a pas rendu.

Exercice 2: Estimation de Pi

L'aire d'un cercle de rayon R est $\pi * R^2$. Si on prend un Rayon de 1, l'aire est donc π . En prenant un cercle de centre 0,0 et de Rayon 1, il est possible d'approximer π .

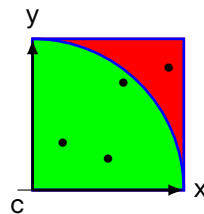


Si on s'intéresse à l'intervalle $[0,1]$ on a donc un quart de cercle et donc une surface de $\frac{\pi}{4}$



La surface du carré (surface verte + surface rouge) est de 1. La surface verte est $\frac{\pi}{4}$.

Si on tire aléatoirement un couple $(x,y) \in [0.0, 1.0]$. Si $x^2 + y^2 \leq 1^2$ (équation de cercle) alors le point est dans le cercle (zone verte), sinon en dehors (zone rouge). La probabilité que l'on soit dans le quart de cercle est de $\frac{\pi}{4}$.



Si on répète l'opération un nombre N (grand) de fois, le rapport entre le nombre de point étant dans le quart de cercle (hit) et le total de points testés (N) approxime $\frac{1}{4}$ de la surface du cercle unitaire.

On a donc $\pi \approx 4 * \frac{hit}{N}$

Ce type d'approche est appelé méthode de Monte-Carlo, ici appliquée à π à l'aide du cercle unitaire.

Variante 1: Programmez cette méthode en monothread

Variante 2: Programmez cette méthode en multithread

Comparez les temps de calculs entre les deux méthodes. Prenez un N suffisamment grand par exemple $N=1\,000\,000$